01 Project Documentation

Explainable Predictive Analytics for Employee Attrition Risk Assessment

Complete technical documentation - 30 pages

Project team	Institutional email
Y Manikanta Sai	2420030082@klh.edu.in
D Sai Srujan	2420030083@klh.edu.in
Golla Rahul	2420040098@klh.edu.in
Boya Vinay	2420090054@klh.edu.in

Department of Computer Science and Information Technology
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad-500043, Telangana, India

Project scope

A full-stack employee-attrition analytics prototype using a benchmark dataset, four machine-learning classifiers, SHAP explanations, a React dashboard, a FastAPI backend and SQLite persistence. This document covers the supplied archive, input and output contracts, recorded results, implementation details and the supplied application screenshot.

Evidence edition: 30 October 2026. Stored outputs and source-defined examples are identified separately; this documentation does not claim a newly executed training or live deployment test.

02 Contents and reading guide

Use pages 3-6 for orientation and setup, pages 7-17 for the data and analytical pipeline, pages 18-22 for API contracts, and pages 23-28 for the user interface. The final pages present your screenshot and the verification and handover checklist.

Pages	Coverage
1-2	Title, project team and document navigation
3	Purpose, objectives and system-wide input/output
4	Archive inventory and file ownership
5	Technology stack and declared dependencies
6	Installation, startup and build procedure
7	Dataset profile and actual stored counts
8-9	Complete 35-column input dictionary
10	Preprocessing and feature transformation
11	Training, candidate models and selection
12	Recorded model metrics and interpretation
13	Confusion matrices and error tradeoffs
14	SHAP global and local explanation outputs
15	Saved artifacts, recommendations and seeding
16-17	Complete database schema and relationships
18	Backend application and API route index
19	Statistics API input/output
20	Employee directory and detail API
21	What-if simulation request and response
22	Retention action create, list and update
23	Frontend shell, navigation and state
24	Executive dashboard inputs and outputs
25	Workforce directory interaction flow
26	Employee detail and explainability view
27	Benchmark display and export
28	Simulation form and retention tracker
29	Supplied screenshot and observed results
30	Verification checklist, limitations and source index

Coverage includes authored source files, configuration, assets, dataset, database, saved ML objects, compiled frontend output, dependency folders and caches. Bundled third-party libraries are described by their dependency role rather than reproduced as source listings.

03 Purpose, scope and overall workflow

The project estimates employee attrition risk and makes the result inspectable through explanatory factors. Its intended use is supportive HR review. The application provides aggregate monitoring, a searchable workforce directory, individual SHAP explanations, model comparisons, scenario editing and retention-action tracking. The project title is preserved exactly throughout this documentation.

System-wide input/output map

Stage	Input	Output
Dataset intake	35-column CSV benchmark	Employee attributes and binary target
Training	30 raw predictors; train/test split	Four classifiers and measured metrics
Prediction	Transformed employee vector	Probability, binary label and display tier
Explanation	Random Forest and transformed records	Global importance and signed local factors
Dashboard	Stored database predictions	KPIs, department and overtime summaries
Simulation	Employee ID and attribute overrides	Re-scored probability, change and SHAP drivers
Action tracking	Employee ID, title, priority, notes	Persisted follow-up record and status

What the project demonstrates

The source connects analytical artifacts with an interactive application. Training and seeding are offline operations. Most screens read stored outputs; simulation performs inference using the serialized model and transformer. The workflow does not predict a resignation date, estimate a causal intervention effect or demonstrate a reduction in real organizational turnover.

Evidence boundaries

The ZIP includes a populated SQLite database and saved model files. Database values reported below are read from that supplied snapshot. Examples of HTTP messages illustrate source-defined contracts unless explicitly identified as stored data. Generated employee identities and prefilled actions are demonstration records.

Primary sources: README.md; backend/ml/train_and_seed.py; frontend/src/App.jsx; backend/api/routers/*.py.

04 Archive inventory and project structure

The retained project extracted from the supplied ZIP contains 12,663 files, including 12,609 files inside node_modules. The authored application is split into backend and frontend directories. The root contains README.md and start_servers.bat.

Area	Files relative to area
API application and routers	main.py routers/employees.py routers/models.py routers/retention.py routers/simulate.py routers/stats.py
ORM and connection	database.py models.py
Training and artifacts	saved_models/best_model.joblib saved_models/metrics.json saved_models/preprocessor.joblib saved_models/shap_explainer.joblib train_and_seed.py
Dataset and database	attrition_data.csv attrition_system.db
UI modules	EmployeeDetailModal.jsx ExecutiveDashboard.jsx ModelBenchmark.jsx Navbar.jsx RetentionTracker.jsx WhatIfSimulator.jsx WorkforceExplorer.jsx
Shell, styles and assets	App.css App.jsx assets/hero.png assets/react.svg assets/vite.svg index.css main.jsx
Frontend setup/build/assets	.gitignore .oxlintrc.json README.md dist/assets/index-Bhv4xpCE.js dist/assets/index-mMgEG0QM.css dist/favicon.svg dist/icons.svg dist/index.html index.html package-lock.json package.json public/favicon.svg public/icons.svg vite.config.js

Generated and external content

frontend/dist contains a built index page and generated JavaScript/CSS bundles. frontend/node_modules contains installed third-party dependencies and executable shims. __pycache__ folders contain compiled Python bytecode. These directories do not replace the editable source, dependency manifests or saved analytical artifacts.

The screenshot shows additional folders such as doc, reports, results and test_rahul in its editor tree. Those names are not present at the top level of the supplied ZIP; this report documents the archive actually provided.

05 Technologies and dependency requirements

Python implements data processing, model fitting, explanations and REST endpoints. React implements the browser interface. SQLAlchemy maps Python entities to a local SQLite file. Vite builds the client and forwards development API requests to FastAPI.

Python requirement	Purpose
fastapi>=0.115.0	REST application
uvicorn>=0.30.0	ASGI development server
sqlalchemy>=2.0.0	Database ORM
pandas>=2.0.0	Tabular input processing
numpy>=1.26.0	Numerical arrays
scikit-learn>=1.5.0	Models and preprocessing
shap>=0.45.0	Feature attribution
joblib>=1.4.0	Artifact serialization
pydantic>=2.8.0	Request schemas
httpx>=0.27.0	HTTP tooling

Frontend package	Declared range	Role
@tailwindcss/vite	^4.3.3	Frontend runtime
lucide-react	^1.63.0	Frontend runtime
react	^19.3.1	Frontend runtime
react-dom	^19.3.1	Frontend runtime
recharts	^3.10.1	Frontend runtime
@types/react	^19.3.0	Frontend development
@types/react-dom	^19.3.0	Frontend development
@vitejs/plugin-react	^6.1.2	Frontend development
autoprefixer	^10.6.2	Frontend development
oxlint	^1.89.0	Frontend development
postcss	^8.5.32	Frontend development
tailwindcss	^4.3.3	Frontend development
vite	^8.3.5	Frontend development

Versions are the ranges declared in the supplied manifests, not independently verified installed runtime versions. The navbar displays a hard-coded SHAP v0.54 label; requirements.txt only specifies shap>=0.45.0. The frontend lockfile records dependency resolution and should be retained for reproducible installation.

06 Installation, startup and build

Prerequisites and working directories

Use Python and Node.js/npm compatible with the supplied dependencies. The archive does not include a fully pinned Python environment or a stated minimum Python/Node version. Keep the backend data and ml/saved_models directories in their relative locations. Perform installation in a copy of the extracted project so the original evidence remains available.

Backend setup - from the extracted project root

```
cd backend
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
python -m uvicorn api.main:app --host 127.0.0.1 --port 8000 --reload
```

The command above binds locally for a development session. The supplied batch file uses 0.0.0.0 instead. API documentation is exposed by FastAPI at http://localhost:8000/docs. The database location is resolved relative to backend/database/database.py.

Frontend setup - in a separate terminal

```
cd frontend
npm ci
npm run dev
```

Vite is configured for port 5173 and proxies /api to http://localhost:8000. npm ci uses the included lockfile. If installation fails, inspect the package engine requirements and the local tool versions before changing the manifests.

Build and maintenance commands

Command	Output / behavior
npm run build	Creates the frontend/dist build
npm run preview	Previews the built frontend; API serving must be arranged
npm run lint	Runs oxlint against the frontend
start_servers.bat	Opens backend and frontend command windows
python ml/train_and_seed.py	Retrains and recreates database tables; existing action records are lost

Do not run the training/seeding command against a database containing actions you need to preserve. Back up the SQLite file first. Vite development proxy settings do not automatically establish a production reverse proxy.

07 Dataset profile and persisted outputs

The CSV has 1,470 records and 35 columns. The target is Attrition, encoded as Yes/No. The code removes four non-model columns and the target, leaving 30 raw predictors. Numerical and categorical transformations then produce the model feature matrix.

CSV property	Observed value
Attrition counts	{'Yes': 237, 'No': 1233}
Department counts	{'Sales': 446, 'Research & Development': 961, 'Human Resources': 63}
Blank CSV values	0
Dataset rows	1,470
Training/test design	80/20 stratified, random_state=42
Held-out records	294

Read-only SQLite snapshot

Table / aggregate	Stored value
employees	1470
predictions	1470
retention_actions	12
model_evaluations	4
Historical attrition rate	16.12%
Mean predicted risk	32.73%
High risk records	203
Low risk records	937
Medium risk records	330

How to interpret the counts

Historical attrition is calculated from known benchmark labels. Mean predicted risk is the average stored model probability, not the historical rate. Risk tiers count model outputs at application thresholds. They do not measure classification accuracy or prove that the individuals will leave. The database contains predictions across the entire benchmark, including training records, whereas the evaluation matrices describe only the held-out test sample.

Sources: backend/data/attrition_data.csv; backend/data/attrition_system.db; backend/ml/train_and_seed.py. Database queried in read-only mode.

08 Input dictionary - fields 1 to 18

The complete source CSV schema is listed over these two pages. Observed ranges describe this file only; they are not automatically enforced by the API. Categorical values are case-sensitive inputs to the trained preprocessing pipeline.

CSV field	Pipeline role	Observed range or values
Age	Numerical	18 to 60
Attrition	Target	No, Yes
BusinessTravel	Categorical	Non-Travel, Travel_Frequently, Travel_Rarely
DailyRate	Numerical	102 to 1499
Department	Categorical	Human Resources, Research & Development, Sales
DistanceFromHome	Numerical	1 to 29
Education	Numerical	1 to 5
EducationField	Categorical	Human Resources, Life Sciences, Marketing, Medical, Other, Technical Degree
EmployeeCount	Excluded	1 to 1
EmployeeNumber	Excluded	1 to 2068
EnvironmentSatisfaction	Numerical	1 to 4
Gender	Categorical	Female, Male
HourlyRate	Numerical	30 to 100
JobInvolvement	Numerical	1 to 4
JobLevel	Numerical	1 to 5
JobRole	Categorical	Healthcare Representative, Human Resources, Laboratory Technician, Manager, Manufacturing Director, Research Director, Research Scientist, Sales Executive, Sales Representative
JobSatisfaction	Numerical	1 to 4
MaritalStatus	Categorical	Divorced, Married, Single

Encoding rules

Attrition=Yes becomes class 1; No becomes class 0. EmployeeNumber is used for display identity but is excluded from model features. EmployeeCount, Over18 and StandardHours are dropped as specified by the training script. The model retains demographic fields, including Gender and Age, so the implementation must not be described as excluding every sensitive attribute.

Example source row (selected fields)

```
{
  "EmployeeNumber": "1",
  "Age": "41",
  "Attrition": "Yes",
  "Department": "Sales",
  "JobRole": "Sales Executive",
  "OverTime": "Yes"
}
```

Education and satisfaction variables are encoded numeric levels in the benchmark. The project does not supply a separate validated dictionary of level descriptions; this report preserves the observed numeric representation.

09 Input dictionary - fields 19 to 35

Compensation, job satisfaction, overtime and experience fields complete the feature set. Currency and geographic units are not established by the supplied training code; numerical values are documented without inferring an organization-specific currency or policy interpretation.

CSV field	Pipeline role	Observed range or values
MonthlyIncome	Numerical	1009 to 19999
MonthlyRate	Numerical	2094 to 26999
NumCompaniesWorked	Numerical	0 to 9
Over18	Excluded	Y
OverTime	Categorical	No, Yes
PercentSalaryHike	Numerical	11 to 25
PerformanceRating	Numerical	3 to 4
RelationshipSatisfaction	Numerical	1 to 4
StandardHours	Excluded	80 to 80
StockOptionLevel	Numerical	0 to 3
TotalWorkingYears	Numerical	0 to 40
TrainingTimesLastYear	Numerical	0 to 6
WorkLifeBalance	Numerical	1 to 4
YearsAtCompany	Numerical	0 to 40
YearsInCurrentRole	Numerical	0 to 18
YearsSinceLastPromotion	Numerical	0 to 15
YearsWithCurrManager	Numerical	0 to 17

Constraints and cross-field consistency

The numerical feature vector includes total working years, years at company, years in current role and years with the current manager. A real inference service should enforce plausible relationships between these fields. The prototype exposes typed simulation fields but does not enforce all such relationships.

Prediction input versus review input

The model consumes the complete transformed predictor set. The interface only edits a subset for simulation. Employee identifiers select a stored row; display names and synthetic email addresses do not enter training. The target Attrition is available for benchmark reporting and is not an inference feature.

Source: backend/data/attrition_data.csv and the explicit drop_cols/cat_cols lists in backend/ml/train_and_seed.py.

10 Preprocessing and feature transformation

Preprocessing is fitted after the stratified train/test split. StandardScaler learns means and scales from training numerical fields. OneHotEncoder learns categorical values, drops the first category, ignores unseen levels and returns a dense array. The fitted transformer processes the test data without refitting.

Input	Operation	Output
Attrition text	Yes mapped to 1	Binary target vector y
35-column dataframe	Drop target and four excluded fields	30-column predictor dataframe X
X and y	train_test_split with stratification	1,176 train / 294 test records
Numerical branch	StandardScaler fit on train	Scaled numerical columns
Categorical branch	OneHotEncoder fit on train	Indicator columns
Both branches	ColumnTransformer concatenation	Consistent transformed feature matrix

Why the fitted transformer is retained

Inference must reuse the exact category ordering, numeric scaling and column order learned during training. A newly fitted encoder or a manually reordered vector could silently change prediction meaning. The code saves preprocessor.joblib alongside the selected model for this reason.

Source implementation excerpt

backend/ml/train_and_seed.py | lines 100-129

```
    print(f"Loaded dataset: {df.shape[0]} rows, {df.shape[1]} columns")

    # Preprocessing
    drop_cols = ["EmployeeCount", "Over18", "StandardHours", "EmployeeNumber"]
    y = (df["Attrition"] == "Yes").astype(int)
    X = df.drop(columns=["Attrition"] + drop_cols)

    cat_cols = ["BusinessTravel", "Department", "EducationField", "Gender", "JobRole",
"MaritalStatus", "OverTime"]
    num_cols = [c for c in X.columns if c not in cat_cols]

    print(f"Numerical features ({len(num_cols)}): {num_cols}")
    print(f"Categorical features ({len(cat_cols)}): {cat_cols}")

    # 80/20 train/test stratified split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    preprocessor = ColumnTransformer([
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(drop="first", sparse_output=False, handle_unknown="ignore"),
cat_cols)
    ])

    X_train_trans = preprocessor.fit_transform(X_train)
    X_test_trans = preprocessor.transform(X_test)
    X_all_trans = preprocessor.transform(X)

    cat_feature_names =
preprocessor.named_transformers_["cat"].get_feature_names_out(cat_cols).tolist()
    all_feature_names = num_cols + cat_feature_names
```

11 Model training and selection

Four classifiers are fitted to the same transformed training set. The source computes label predictions and positive-class probabilities on the held-out set. It measures accuracy, precision, recall, F1, ROC-AUC and confusion matrices. Precision/recall calculations use zero_division=0 to keep metric output defined when no positives are predicted.

Candidate	Key supplied parameters
Logistic Regression	{"solver": "lbfgs", "class_weight": "balanced", "max_iter": 1000}
Decision Tree	{"criterion": "gini", "max_depth": 5, "class_weight": "balanced"}
Random Forest	{"n_estimators": 160, "max_depth": 8, "class_weight": "balanced"}
Gradient Boosting	{"n_estimators": 130, "learning_rate": 0.08, "max_depth": 4}

Selection behavior

best_name is assigned to Random Forest before model training. The source does not perform a hyperparameter search or select the winner by maximizing the reported F1 or ROC-AUC. Random Forest is the deployed implementation choice; Gradient Boosting leads on accuracy/AUC, and Logistic Regression leads on recall/F1 in the saved results.

Training outputs

The fitted Random Forest, fitted preprocessor and SHAP explainer are serialized. The metrics file records the candidate results, global importance, SHAP baseline, total records and timestamp. The routine also calculates full-dataset predictions and recreates the application database.

Source implementation excerpt

backend/ml/train_and_seed.py | lines 156-178

```
    best_name = "Random Forest"  # Balanced precision/recall and fast TreeSHAP support
    best_f1 = -1

    for name, cfg in candidate_models.items():
        m = cfg["model"]
        m.fit(X_train_trans, y_train)
        trained_models[name] = m

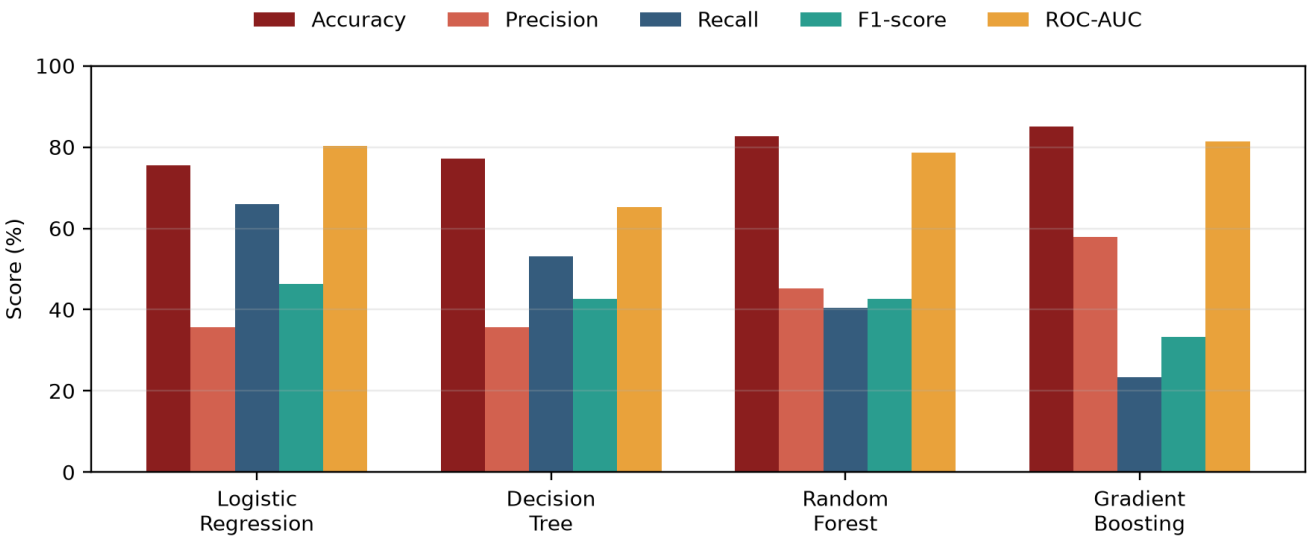
        preds = m.predict(X_test_trans)
        probs = m.predict_proba(X_test_trans)[:, 1]

        acc = float(accuracy_score(y_test, preds))
        prec = float(precision_score(y_test, preds, zero_division=0))
        rec = float(recall_score(y_test, preds, zero_division=0))
        f1 = float(f1_score(y_test, preds, zero_division=0))
        auc = float(roc_auc_score(y_test, probs))
        cm = confusion_matrix(y_test, preds)

        cm_dict = {
            "tn": int(cm[0, 0]),
            "fp": int(cm[0, 1]),
            "fn": int(cm[1, 0]),
            "tp": int(cm[1, 1])
```

12 Recorded evaluation results

The following graph and table reproduce the saved metrics. They are reported outputs from the archive, not a fresh retraining result. All scores are percentages and refer to the 294-record stratified holdout.



Model	Accuracy	Precision	Recall	F1	AUC
Logistic Regression	75.51	35.63	65.96	46.27	80.35
Decision Tree	77.21	35.71	53.19	42.74	65.17
Random Forest	82.65	45.24	40.43	42.70	78.55
Gradient Boosting	85.03	57.89	23.40	33.33	81.46

Interpretation

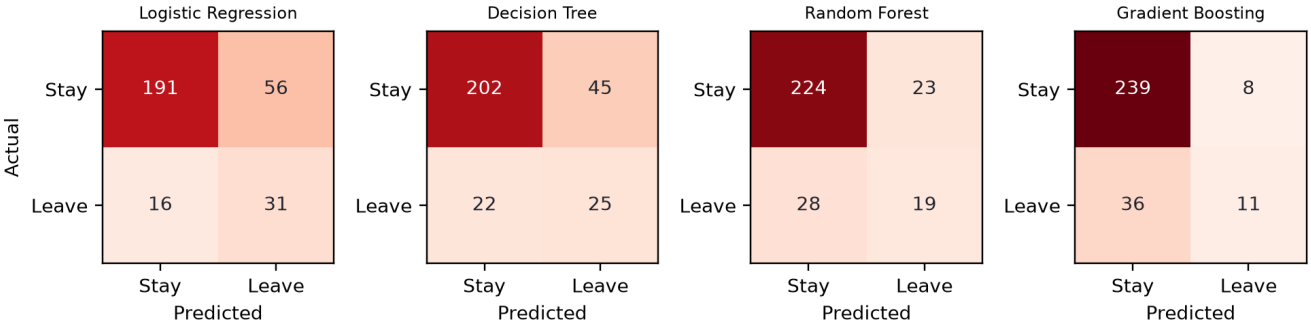
Gradient Boosting records the greatest accuracy and ROC-AUC but detects only 23.40% of the recorded positive class at its classification threshold. Logistic Regression detects 65.96% while producing more false alarms. Random Forest has 40.43% recall and 45.24% precision. A model choice must account for these different operating points.

Consistency with the presentation

The stored metrics differ from some values printed in the PPT and README. This document uses metrics.json and the confusion-matrix arithmetic: Decision Tree precision is 35.71%, and Gradient Boosting F1 is 33.33%. Neither the PPT value of 48.35% for boosting F1 nor the claim that Random Forest wins every measure is supported by the saved matrices.

Source: backend/ml/saved_models/metrics.json; model_evaluations table.

13 Confusion matrices and error analysis



Rows are actual outcomes and columns are predicted outcomes. Stay is the negative class and Leave is the positive class. Each matrix totals 294 records: 247 stayers and 47 leavers.

Model	TN	FP	FN	TP	Review flags
Logistic Regression	191	56	16	31	87
Decision Tree	202	45	22	25	70
Random Forest	224	23	28	19	42
Gradient Boosting	239	8	36	11	19

Majority baseline

Predicting Stay for every record would yield 247/294 = 84.01% accuracy but zero positive recall. This explains why accuracy alone is insufficient. Random Forest falls below that accuracy baseline while detecting 19 actual leavers. Gradient Boosting exceeds it only slightly but detects 11.

Review workload

Random Forest flags 42 records: 19 actual positives and 23 false positives. It misses 28 actual positives. Logistic Regression flags 87 records and detects 31 positives. These counts describe potential review workload, not the actual cost or benefit of intervention.

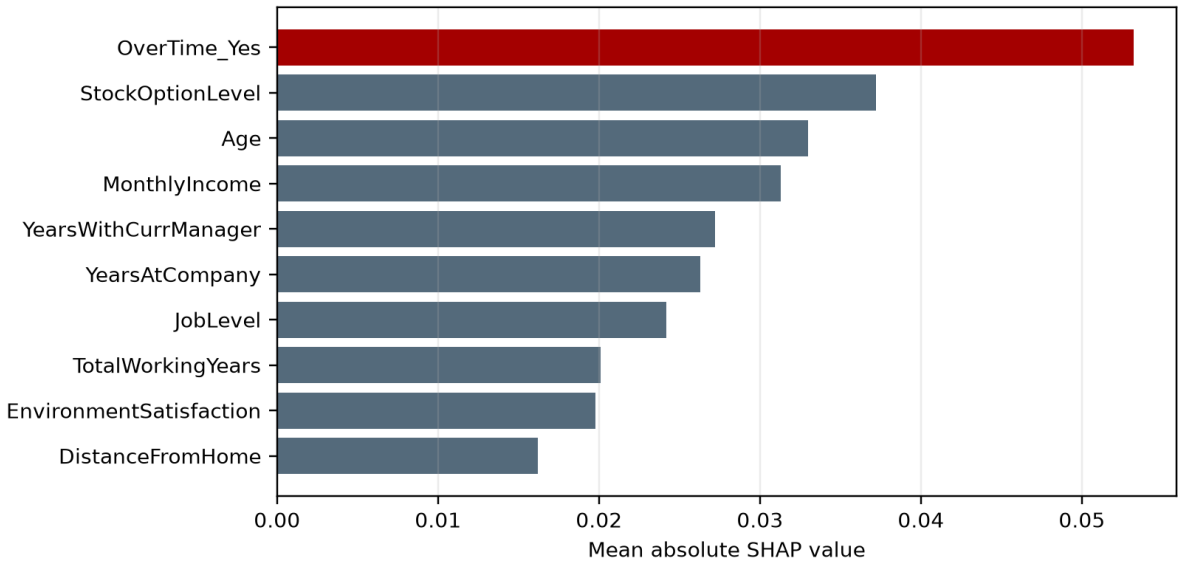
Metric equations

```
Accuracy = (TP + TN) / N
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 = 2TP / (2TP + FP + FN)
Specificity = TN / (TN + FP)
Balanced accuracy = (Recall + Specificity) / 2
```

ROC-AUC requires probability rankings across thresholds. The archive provides the scalar AUC values; a single confusion matrix cannot reconstruct a genuine ROC curve.

14 SHAP explanations and analytical outputs

TreeExplainer is constructed from the saved Random Forest. The script handles list-based and three-dimensional SHAP return formats, selecting the positive-class contributions. Explanations are computed for all 1,470 records using the fitted transformation.



Global output

The global ranking is mean(abs(SHAP)) per transformed feature. Overtime_Yes is first at 0.0532, followed by StockOptionLevel, Age, MonthlyIncome and YearsWithCurrManager. Magnitude does not preserve sign: a global value is not proof that a feature raises every employee score by that amount.

Local output

Each employee stores up to five positive and five negative factors. Each factor records a transformed feature name, signed SHAP value and absolute magnitude. The selected top factors omit smaller contributions, so the visible list need not reconstruct the entire prediction. The baseline is an explainer reference, not necessarily the historical attrition rate.

Consumer mapping

Output	Consumer
global_importance array	ModelBenchmark and /api/models/global-importance
top_positive_factors	Employee detail risk drivers
top_negative_factors	Employee detail protective factors
top_simulated_drivers	What-if result panel

15 Artifacts, recommendations and database seeding

Artifact	Size	SHA-256 prefix
best_model.joblib	2,288,025 bytes	9a685dfcc4f1d14c39f1
metrics.json	3,269 bytes	2a6d8a2c9057ac51ed9a
preprocessor.joblib	6,413 bytes	1c69635c1bdfc79d3394
shap_explainer.joblib	3,402,742 bytes	1ba4f7f71a7a15ca6a78

The model, preprocessing and explainer form one compatible artifact set. Loading mismatched files risks incorrect predictions or mislabelled explanations. Joblib artifacts should only be loaded from trusted project sources. The file hashes above identify this supplied snapshot; archive provenance is recorded on page 30.

Rule-based recommendations

Trigger examined in source	Suggested activity
Overtime value or driver	Workload rebalancing
Low income or income driver	Compensation review
Low satisfaction or satisfaction driver	Career-development check-in
Work-life balance level	Flexible-hours discussion
Long time in current role	Internal mobility review
Environment satisfaction	Workplace feedback review
No matching rules	Regular pulse and recognition check-ins

The routine returns at most three recommendations. These are predefined rules, not learned treatment effects. It checks OverTime while transformed SHAP names use OverTime_Yes; the additional raw overtime check still provides a trigger.

Seeding behavior

The routine drops and recreates ORM tables, saves model evaluations, generates display names and synthetic employee emails, adds predictions and seeds up to twelve example actions for high-risk records. A Completed seed action is demonstration data and does not prove that an intervention occurred.

Sources: backend/ml/train_and_seed.py; backend/ml/saved_models/*.joblib; metrics.json.

16 Database schema - employees

SQLite stores the full application state in backend/data/attrition_system.db. The Employee ORM entity contains display identity, raw benchmark attributes and relationships to predictions and retention actions. The field list below is read directly from the supplied database schema.

Column	SQLite type	Constraint
id	INTEGER	PK
employee_number	VARCHAR(50)	optional
full_name	VARCHAR(120)	required
email	VARCHAR(120)	optional
avatar_url	VARCHAR(255)	optional
age	INTEGER	required
gender	VARCHAR(20)	required
marital_status	VARCHAR(30)	required
distance_from_home	INTEGER	required
education	INTEGER	required
education_field	VARCHAR(50)	required
department	VARCHAR(80)	required
job_role	VARCHAR(80)	required
job_level	INTEGER	required
business_travel	VARCHAR(50)	required
job_involvement	INTEGER	required
monthly_income	FLOAT	required
daily_rate	INTEGER	required
hourly_rate	INTEGER	required
monthly_rate	INTEGER	required
percent_salary_hike	INTEGER	required
stock_option_level	INTEGER	required
over_time	VARCHAR(10)	required
work_life_balance	INTEGER	required
environment_satisfaction	INTEGER	required
job_satisfaction	INTEGER	required
relationship_satisfaction	INTEGER	required
performance_rating	INTEGER	required
total_working_years	INTEGER	required
years_at_company	INTEGER	required
years_in_current_role	INTEGER	required
years_since_last_promotion	INTEGER	required
years_with_curr_manager	INTEGER	required
num_companies_worked	INTEGER	required
training_times_last_year	INTEGER	required
actual_attrition	VARCHAR(10)	required

Employee.prediction is one-to-one; retention_actions is one-to-many. ORM cascade settings include delete-orphan. Synthetic names and emails are display metadata. Foreign-key enforcement should be checked independently before production use.

17 Database schema - analytical and action tables

Predictions

Column	Type	Constraint
id	INTEGER	PK
employee_id	INTEGER	required
attrition_probability	FLOAT	required
predicted_label	INTEGER	required
risk_category	VARCHAR(20)	required
shap_base_value	FLOAT	optional
top_positive_factors	JSON	optional
top_negative_factors	JSON	optional
retention_recommendations	JSON	optional
assessed_at	DATETIME	optional

Retention actions

Column	Type	Constraint
id	INTEGER	PK
employee_id	INTEGER	required
action_type	VARCHAR(100)	required
title	VARCHAR(200)	required
priority	VARCHAR(20)	optional
status	VARCHAR(30)	optional
assigned_hr	VARCHAR(100)	optional
notes	TEXT	optional
target_date	VARCHAR(50)	optional
created_at	DATETIME	optional
updated_at	DATETIME	optional

Model evaluations

id (INTEGER); model_name (VARCHAR(80)); model_type (VARCHAR(50)); accuracy (FLOAT); precision (FLOAT); recall (FLOAT); f1_score (FLOAT); roc_auc (FLOAT); confusion_matrix (JSON); hyperparameters (JSON); is_best_model (BOOLEAN); trained_at (DATETIME)

Prediction.employee_id is unique, linking one stored prediction to each employee. RetentionAction.employee_id allows multiple actions. JSON columns preserve factor arrays, recommendations, confusion matrices and parameters. SQLAlchemy creates a session per dependency call and closes it in a finally block.

18 Backend application and API index

api/main.py creates a FastAPI application, registers five routers and enables development cross-origin access. Route handlers perform synchronous database queries or model inference. The application title matches the project title; the API version is 1.0.0.

Method	Endpoint	Function
GET	/api/health	Static service status
GET	/api/stats	Dashboard aggregates
GET	/api/employees	Filtered, paginated directory
GET	/api/employees/filters	Distinct UI filter choices
GET	/api/employees/{employee_id}	Employee detail and explanations
GET	/api/models/evaluations	Stored model comparison
GET	/api/models/global-importance	metrics.json explanation ranking
POST	/api/simulate	What-if inference
GET	/api/retention-actions	Action list and filters
POST	/api/retention-actions	Create action
PATCH	/api/retention-actions/{action_id}	Update action fields

Health output defined in source

```
{
  "status": "healthy",
  "service": "Explainable HR Analytics API",
  "version": "1.0.0",
  "dataset": "IBM HR Analytics Employee Attrition & Performance",
  "records": 1470
}
```

The health endpoint returns static metadata and does not verify database freshness, model compatibility or actual record counts. No authentication dependency is registered on the supplied routes. CORS allows all origins, methods and headers with credentials configured, which is development configuration rather than a security boundary.

Source implementation excerpt

backend/api/main.py | lines 18-33

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(stats.router)
app.include_router(employees.router)
app.include_router(simulate.router)
app.include_router(models.router)
app.include_router(retention.router)

@app.get("/api/health")
```

19 Statistics API - inputs and outputs

GET /api/stats has no filter parameters. It derives aggregate data from employees and predictions. The frontend fetches this response on load and when Refresh Metrics is selected. Historical and predicted quantities are kept in separate response fields.

Response field	Calculation / content
kpi.total_employees	Employee table row count
kpi.high_risk_count	Predictions with risk_category High
kpi.medium_risk_count / low_risk_count	Counts for corresponding stored tiers
kpi.historical_attrition_rate	100 * actual Attrition Yes count / employee count
kpi.average_risk_score	100 * average stored probability
risk_distribution	Tier labels, counts, colors and keys
department_analytics	Headcount, actual rate, high count, mean risk
overtime_analytics	Overtime and standard-hours cohorts
top_vulnerable_roles	Six roles ordered by average predicted risk

Stored-value illustration of KPI fields

```
{
  "total_employees": 1470,
  "historical_attrition_rate": 16.12,
  "average_risk_score": 32.73,
  "high_risk_count": 203,
  "low_risk_count": 937,
  "medium_risk_count": 330
}
```

This illustration is calculated from the archived SQLite database using the same aggregate definitions; it is not captured from a newly running HTTP service. Zero-count cases return zero rates in the source. Department and role values summarize the full benchmark, including training records.

Source implementation excerpt

backend/api/routers/stats.py | lines 12-27

```
    # Historical Attrition Rate
    attrition_yes = db.query(Employee).filter(Employee.actual_attrition == "Yes").count()
    historical_attrition_rate = round((attrition_yes / total_employees) * 100, 2) if
total_employees else 0.0

    # Risk Categories from Predictions
    high_risk_count = db.query(Prediction).filter(Prediction.risk_category == "High").count()
    medium_risk_count = db.query(Prediction).filter(Prediction.risk_category ==
"Medium").count()
    low_risk_count = db.query(Prediction).filter(Prediction.risk_category == "Low").count()

    # Average Predicted Probability
    avg_risk_score = db.query(func.avg(Prediction.attrition_probability)).scalar() or 0.0
    avg_risk_score = round(float(avg_risk_score) * 100, 2)

    # Risk Distribution Breakdown
    risk_distribution = [
```

20 Employee API - input and output contracts

The directory route joins Employee with Prediction, filters records and returns a page. The detail route includes raw values, signed explanation lists and linked actions. An absent detail identifier produces HTTP 404 with Employee not found.

Query input	Default / constraints
page	1; minimum 1
page_size	12; range 1-100; frontend requests 10
department / job_role	Optional exact filters; All disables
risk_category / over_time	Optional exact filters; All disables
search	Matches name, number, email or job role
sort_by	probability, monthly_income, age, years_at_company, name
sort_order	desc or asc

Example requests

```
GET /api/employees?page=1&page_size=10&risk_category=High
GET /api/employees/filters
GET /api/employees/1
```

Response envelopes

Route	Output groups
Directory	items, total, page, page_size, total_pages
Filter choices	departments, job_roles, risk_categories, overtime_options
Employee detail	id, identity, demographics, job, compensation, workload_satisfaction, tenure, ground_truth, prediction, retention_actions

Read-only stored example

```
{
  "id": 1,
  "department": "Sales",
  "job_role": "Sales Executive",
  "over_time": "Yes",
  "monthly_income": 5993.0
}
```

The directory exposes actual_attrition alongside modeled risk as a benchmark demonstration. A live employee view would need a different policy for unavailable future ground truth and for demographic data exposure.

21 Simulation API - input and output contracts

POST /api/simulate accepts JSON. employee_id selects a stored profile. Optional overrides replace selected values in a copy, then the fitted preprocessor and Random Forest produce a new probability. The source does not persist the simulated profile.

Optional request field	Type
employee_id	Integer
over_time	String
monthly_income	Number
work_life_balance, job_satisfaction	Integer
environment_satisfaction, stock_option_level	Integer
distance_from_home, job_involvement	Integer
percent_salary_hike, years_in_current_role	Integer
custom_overrides	Dictionary of additional feature overrides

Illustrative input - not a measured intervention

```
{
  "employee_id": 1,
  "over_time": "No",
  "monthly_income": 6500
}
```

Response key	Meaning
original_risk_score / original_category	Stored baseline scaled to percent and tier
simulated_risk_score / simulated_category	Fresh model score and application tier
risk_delta	Signed change in percentage points
risk_reduction_percentage	Baseline minus simulated, in percentage points
top_simulated_drivers	Six largest absolute SHAP factors
applied_adjustments	Changed raw fields and new values

No numerical response is invented here. Without an employee ID, the source uses an illustrative profile and a fixed 0.55 baseline. EmployeeNotFound returns 404. Input bounds and cross-field plausibility are not fully validated; custom overrides can create unsupported combinations. New SHAP labels should be checked against transformer column order.

22 Retention API - input and output contracts

The action API stores planned human activities linked to an employee. It does not deliver a message or perform an intervention. GET supports status and priority filters; results include employee identity and role, action properties and timestamps.

Illustrative create input

```
POST /api/retention-actions
{
  "employee_id": 1,
  "action_type": "Workload & Overtime Reduction",
  "title": "Review overtime allocation",
  "priority": "High",
  "status": "Planned",
  "assigned_hr": "HR Retention Partner",
  "notes": "Discuss a feasible workload adjustment",
  "target_date": "Next review meeting"
}
```

Operation	Output / error
POST existing employee	message: Retention action created; action_id
POST missing employee	404 Employee not found
GET list	actions array and total
PATCH existing action	message and updated status
PATCH missing action	404 Retention action not found

Illustrative update input

```
PATCH /api/retention-actions/{action_id}
{"status": "In Progress", "notes": "Review scheduled"}
```

Creation requires employee_id, action_type and title. Update fields are optional and include status, priority, assigned_hr, notes and target_date. The update sets updated_at to the current UTC time. Status strings are not constrained to a strict enumeration by the request schema, and target_date is stored as text.

Source implementation excerpt

backend/api/routers/retention.py | lines 11-27

```
class ActionCreate(BaseModel):
    employee_id: int
    action_type: str
    title: str
    priority: str = "Medium"
    status: str = "Planned"
    assigned_hr: str = "HR Retention Partner"
    notes: Optional[str] = None
    target_date: Optional[str] = None

class ActionUpdate(BaseModel):
    status: Optional[str] = None
    priority: Optional[str] = None
    assigned_hr: Optional[str] = None
    notes: Optional[str] = None
    target_date: Optional[str] = None
```

23 Frontend shell, navigation and state

main.jsx mounts App inside React StrictMode and imports the global stylesheet. App.jsx controls navigation through activeTab rather than URL-based routing. The selected employee identifier is shared between the detail view and simulator. Directory filters can be set by dashboard navigation callbacks.

Tab identifier	Rendered component / view
dashboard	ExecutiveDashboard
directory	WorkforceExplorer
xai	SHAP workspace with EmployeeDetailModal
simulator	WhatIfSimulator
benchmarks	ModelBenchmark
retention	RetentionTracker

Shared state

activeTab defaults to dashboard. selectedEmployeeId defaults to 1. isModalOpen controls the overlay. directoryRiskFilter and directoryOvertimeFilter default to All. Opening an employee sets the selected ID and opens the modal. Opening simulation sets the selected ID and changes the active tab. Dashboard cohort actions set filter state and switch to the directory.

Source implementation excerpt

frontend/src/App.jsx | lines 10-34

```
export default function App() {
  const [activeTab, setActiveTab] = useState('dashboard');
  const [selectedEmployeeId, setSelectedEmployeeId] = useState(1);
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [directoryRiskFilter, setDirectoryRiskFilter] = useState('All');
  const [directoryOvertimeFilter, setDirectoryOvertimeFilter] = useState('All');

  // Navigation callbacks
  const handleOpenEmployeeDetail = (empId) => {
    setSelectedEmployeeId(empId);
    setIsModalOpen(true);
  };

  const handleOpenSimulator = (empId) => {
    setSelectedEmployeeId(empId);
    setActiveTab('simulator');
  };

  const handleNavigateToDirectory = (riskFilter = 'All', otFilter = 'All') => {
    setDirectoryRiskFilter(riskFilter);
    setDirectoryOvertimeFilter(otFilter);
    setActiveTab('directory');
  };

  return (
```

Styles, static assets and configuration

index.css and App.css provide styles; Tailwind is installed through the Vite plugin. The archive also retains React/Vite/hero assets and public favicon/icons assets. index.html provides the root mount container. .gitignore excludes development outputs as configured; .oxlintrc.json configures linting. These support files are part of the package but are not analytical inputs.

Source: frontend/src/main.jsx, App.jsx, components/Navbar.jsx, vite.config.js.

24 Executive dashboard - input and output

ExecutiveDashboard requests /api/stats and renders four main cards plus risk, overtime, department and role summaries. It manages loading and refresh state locally. Dashboard controls navigate into prefiltered workforce cohorts.

Display	Source field / interaction
Total Active Workforce	kpi.total_employees
High Flight Risk	kpi.high_risk_count and share of total
Historical Attrition Rate	kpi.historical_attrition_rate
Average Predicted Flight Risk	kpi.average_risk_score
Risk category bars	risk_distribution counts
Overtime comparison	overtime_analytics
Department table	department_analytics
Vulnerable job roles	top_vulnerable_roles
Review urgent cohort	Directory with High tier
Filter overtime risk	Directory with High tier and Yes overtime

Input-to-display sequence

On mount, fetchStats sets loading, requests JSON and stores stats. A finally block clears the loading state. The Refresh Metrics button repeats the fetch. If stats is absent, the component presents the loading panel. Failed fetches are logged to the console; a distinct user-facing error state is limited.

Claims that require interpretation

The source contains fixed text describing overtime workers as 3.2 times more prone to departure and suggesting up to 45 annual exits could be prevented. Those sentences are not calculated from the response and are not evidence of intervention outcomes. The label Calibrated ensemble model probability also does not establish that calibration was performed.

Source implementation excerpt

frontend/src/components/ExecutiveDashboard.jsx | lines 14-36

```
export default function ExecutiveDashboard({ onNavigateToDirectory, onSelectEmployee }) {
  const [stats, setStats] = useState(null);
  const [loading, setLoading] = useState(true);

  const fetchStats = async () => {
    try {
      setLoading(true);
      const res = await fetch('/api/stats');
      const data = await res.json();
      setStats(data);
    } catch (err) {
      console.error('Failed to load stats:', err);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchStats();
  }, []);

  if (loading || !stats) {
    return (
```

25 Workforce directory - user workflow

WorkforceExplorer provides the searchable employee table. It loads filter options once, builds query parameters from current state and fetches a page of records. The source fixes the interface page size to 10, while the API allows 1-100.

User input	Effect	Visible output
Search text	Query name/number/email/role	Matching employee rows
Department / role	Restrict matching cohort	Filtered counts and rows
Risk tier / overtime	Apply model/cohort filters	Selected review group
Sort field and direction	Change server order	Reordered employee rows
Previous / next	Change requested page	Updated page and totals
Open employee	Callback with employee ID	Detailed explanation view
Open simulator	Callback with employee ID	Scenario form for that record

Typical use

Start with the full directory, choose a department or a risk tier, then inspect the leading factor before opening an employee. The table includes risk score, category and contextual values. A displayed high score is a review cue rather than a conclusion about intention. Resetting filters should restore the full cohort, and page numbers should remain valid when the cohort changes.

Source implementation excerpt

frontend/src/components/WorkforceExplorer.jsx | lines 22-52

```
  const [employees, setEmployees] = useState([]);
  const [total, setTotal] = useState(0);
  const [totalPages, setTotalPages] = useState(1);
  const [page, setPage] = useState(1);
  const [pageSize] = useState(10);
  const [loading, setLoading] = useState(true);

  // Filters
  const [search, setSearch] = useState('');
  const [department, setDepartment] = useState('All');
  const [jobRole, setJobRole] = useState('All');
  const [riskCategory, setRiskCategory] = useState(initialRiskFilter);
  const [overTime, setOverTime] = useState(initialOvertimeFilter);
  const [sortBy, setSortBy] = useState('probability');
  const [sortOrder, setSortOrder] = useState('desc');

  // Filter options
  const [departments, setDepartments] = useState(['All']);
  const [roles, setRoles] = useState(['All']);

  useEffect(() => {
    fetch('/api/employees/filters')
      .then(res => res.json())
      .then(data => {
        setDepartments(data.departments || ['All']);
        setRoles(data.job_roles || ['All']);
      })
      .catch(err => console.error(err));
  }, []);

  const fetchEmployees = async () => {
```

The server joins only employees with predictions. Therefore, missing prediction rows can make the directory count differ from the total employee table count. That difference should be investigated rather than interpreted as a low-risk result.

26 Employee detail and explainability view

EmployeeDetailModal fetches /api/employees/{id}. It groups demographics, job, compensation, satisfaction and tenure information with a modeled score and SHAP explanations. The same component is used by the overlay and the dedicated XAI workspace.

Input / stored group	Interface output
Employee identifier	Fetch selected profile
prediction.risk_score_pct	Risk percentage and category
top_positive_factors	Largest positive contribution list
top_negative_factors	Largest protective contribution list
recommendations	Rule-generated supportive suggestions
retention_actions	Existing action history
Form title, action type, priority, notes	POST request to create action
Open simulator callback	Transfer selected employee ID

Creating a supportive action

The form defaults to Workload & Overtime Reduction with High priority. A submitted title, type, priority and notes accompany the current employee ID. Success updates local state. The request writes a database record; it does not contact the employee or prove that an HR meeting occurred.

Source implementation excerpt

frontend/src/components/EmployeeDetailModal.jsx | lines 20-45

```
export default function EmployeeDetailModal({ employeeId, onClose, onOpenSimulator }) {
  const [emp, setEmp] = useState(null);
  const [loading, setLoading] = useState(true);
  const [currentId, setCurrentId] = useState(employeeId);

  // New action form state
  const [showActionForm, setShowActionForm] = useState(false);
  const [actionTitle, setActionTitle] = useState('');
  const [actionType, setActionType] = useState('Workload & Overtime Reduction');
  const [priority, setPriority] = useState('High');
  const [notes, setNotes] = useState('');
  const [submittingAction, setSubmittingAction] = useState(false);
  const [actionSuccess, setActionSuccess] = useState(false);

  const fetchDetail = async (id) => {
    try {
      setLoading(true);
      const res = await fetch(`/api/employees/${id}`);
      if (!res.ok) throw new Error('Employee not found');
      const data = await res.json();
      setEmp(data);
    } catch (err) {
      console.error(err);
    } finally {
      setLoading(false);
    }
```

Interpretation checks

The largest signed factors describe the fitted model. A short top-factor list is not the full SHAP decomposition. Synthetic display identity should not be mistaken for a real participant. Actual attrition is benchmark ground truth and would not be known in advance for a current employee. Reviewers should distinguish that recorded label from the prediction.

Source: frontend/src/components/EmployeeDetailModal.jsx and backend/api/routers/employees.py.

27 Model benchmark, global importance and export

ModelBenchmark requests evaluations and global importance concurrently. It tracks the selected model name to update the displayed confusion matrix. A JSON export action packages the benchmark evidence for download.

Source endpoint	Output used
/api/models/evaluations	Model rows, metrics, confusion matrices and selection label
/api/models/global-importance	Global SHAP ranking, baseline and record count

Data-source distinction

Evaluations come from the model_evaluations database table. Global importance is loaded from metrics.json. If the file is absent, the global endpoint returns an empty importance list with fallback metadata. Thus, a successful HTTP response does not necessarily mean that explanation artifacts are present. The UI and the report should distinguish an empty result from measured zero importance.

Source implementation excerpt

frontend/src/components/ModelBenchmark.jsx | lines 14-56

```
export default function ModelBenchmark() {
  const [benchmarkData, setBenchmarkData] = useState(null);
  const [globalImportance, setGlobalImportance] = useState([]);
  const [selectedModelName, setSelectedModelName] = useState('Random Forest');
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    Promise.all([
      fetch('/api/models/evaluations').then(res => res.json()),
      fetch('/api/models/global-importance').then(res => res.json())
    ])
      .then(([evals, gImp]) => {
        setBenchmarkData(evals);
        setGlobalImportance(gImp.global_importance || []);
      })
      .catch(err => console.error(err))
      .finally(() => setLoading(false));
  }, []);

  if (loading || !benchmarkData) {
    return (
      <div className="py-20 text-center text-slate-400">
        <BarChart3 className="w-8 h-8 animate-spin mx-auto text-indigo-500 mb-3" />
        <p className="text-sm">Loading Research Paper Benchmark Metrics...</p>
      </div>
    );
  }

  const selectedModel = benchmarkData.evaluations.find(m => m.model_name === selectedModelName)
|| benchmarkData.evaluations[0];

  const handleExportJSON = () => {
    const jsonStr = JSON.stringify(benchmarkData, null, 2);
    const blob = new Blob([jsonStr], { type: 'application/json' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = 'XAI_Attrition_Research_Paper_Results.json';
    a.click();
  };

  return (
    <div className="space-y-6">
      {/* Title & Research Paper Callout */}
```

Expected output

The table permits comparison of accuracy, precision, recall, F1 and ROC-AUC. Selecting a model displays TN, FP, FN and TP counts. The global ranking pertains to Random Forest regardless of which confusion matrix is selected. The hard-coded selected_best_model field in the API is Random Forest and should not be confused with automated best-score selection.

Source: ModelBenchmark.jsx; backend/api/routers/models.py; metrics.json.

28 Simulator form and retention tracker

WhatIfSimulator initializes form controls from the selected employee detail response. The input state includes overtime, monthly income, work-life balance, job and environment satisfaction, stock-option level, distance from home and salary-hike percentage. Reset restores the fetched baseline values.

Automatic scenario request

Changing a listed control triggers runSimulation through a React effect after employee data is loaded. The form POSTs JSON to /api/simulate and displays baseline score, simulated score, tier, change and top drivers. The UI exposes fewer fields than the backend schema: job_involvement and years_in_current_role are API options but are not among these eight form states.

Source implementation excerpt

frontend/src/components/WhatIfSimulator.jsx | lines 62-85

```
      setSimulating(true);
      const res = await fetch('/api/simulate', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          employee_id: employeeId,
          over_time: overTime,
          monthly_income: monthlyIncome,
          work_life_balance: workLifeBalance,
          job_satisfaction: jobSatisfaction,
          environment_satisfaction: environmentSatisfaction,
          stock_option_level: stockOptionLevel,
          distance_from_home: distanceFromHome,
          percent_salary_hike: percentSalaryHike
        })
      });
      const data = await res.json();
      setSimResult(data);
    } catch (err) {
      console.error(err);
    } finally {
      setSimulating(false);
    }
  };
```

Retention tracker input/output

RetentionTracker requests /api/retention-actions with optional status and priority query parameters. Changing either filter refreshes the list. PATCH sends the chosen status to an action-specific endpoint; on success, the local array is updated. Clicking an employee opens the detail view through the callback.

Interaction	Expected output
Change simulator control	Updated model result after request
Reset scenario	Original form values restored
Filter action status/priority	Filtered action cards
Update action status	Stored and locally displayed status
Open linked employee	Profile and explanation context

Concurrent simulation requests are not cancelled by the shown effect. Rapid input changes can allow older responses to arrive later. A future improvement is request sequencing or cancellation, plus explicit user-visible API failure states.

29 Supplied screenshot - application evidence

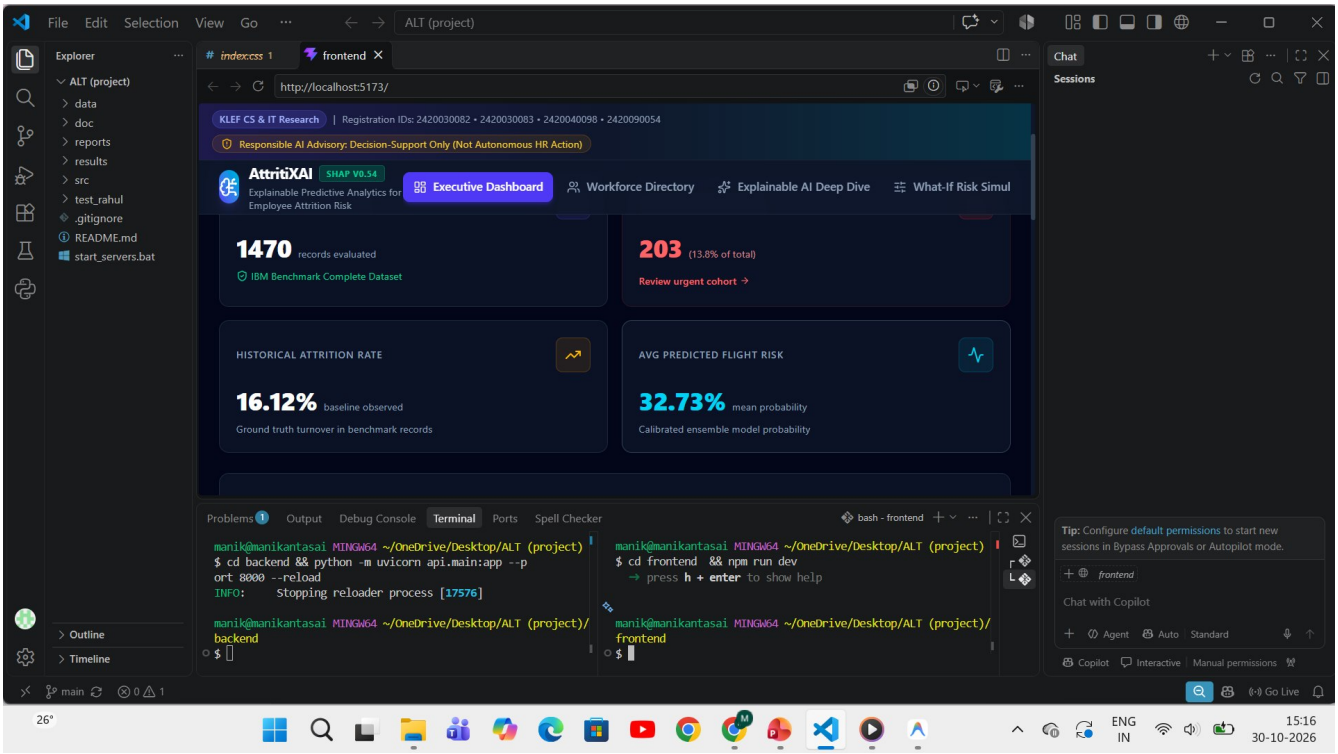


Figure: Original user-supplied screenshot, included without altering its content. It shows the AttritiXAI interface open at localhost:5173 inside VS Code, the navigation banner and four KPI cards.

Visible item	Displayed value / observation
Records evaluated	1,470
High-risk count	203 (13.8% of total)
Historical attrition rate	16.12%
Average predicted risk	32.73%
Application context	Executive Dashboard; KLEF CS & IT banner
Backend terminal	Shows a stopping reloader message and a prompt
Frontend terminal	Shows npm run dev history and a prompt

How this screenshot is used

The picture documents a displayed application state. The terminal messages do not establish that both services were still running when the screenshot was taken. The metrics are reported as visible screen values, separately from the archived SQLite snapshot. Any difference between the picture and the ZIP can reflect another run or project copy.

Source alignment

The editor tree includes folders not present at the root of the supplied archive. This image is included as user-provided evidence, not as proof that those additional folders were inspected. The hard-coded SHAP version badge and calibration wording remain part of the original screenshot; neither substitutes for dependency or calibration verification.

Screenshot supplied by the project team on 30 October 2026. No fabricated application screenshots are used.

30 Verification, limitations and handover

Documentation checks completed: archive inventory, authored-source inspection, CSV structure and value ranges, read-only database counts/schema, saved metrics and matrix arithmetic, route/field mapping and screenshot interpretation. The application was not retrained or launched as part of this documentation task.

Recommended functional check	Expected result
Unchanged employee simulation	Same score under identical artifacts
Model/preprocessor/explainer compatibility	Matching transformed dimensions and feature labels
Full SHAP reconstruction	Baseline plus all contributions matches explained output
Missing employee/action ID	404 with source-defined message
Directory pagination and filters	Consistent counts and valid page bounds
Action create/update/read-back	Persisted values match the submitted fields
Empty/missing metrics file	Clear missing-evidence state
Frontend build and lint	Commands finish successfully in a compatible environment
Prediction tier boundaries	Low <0.30; Medium 0.30 to <0.60; High >=0.60
Artifact and data freshness	Version/timestamp provenance remains consistent

Known limitations

The model includes sensitive demographic inputs; no subgroup fairness evaluation is supplied. Classification uses a different threshold from the High display tier. Probability calibration is not demonstrated. Default simulation uses an illustrative fixed baseline; reported changes are percentage points. There is no demonstrated authentication layer, production security configuration, prospective intervention study or repeat-split generalization analysis.

Primary source index

ALT (project).zip: README.md; start_servers.bat; backend/requirements.txt; backend/api/main.py and routers; backend/database; backend/ml/train_and_seed.py and saved_models; backend/data; frontend/package.json and package-lock.json; vite.config.js; frontend/src; frontend/public; frontend/dist; bundled node_modules and caches. The PPT provides project identity and presentation context; the supplied screenshot provides visual evidence.

Archive provenance: Original ZIP no longer at the supplied Downloads path; inspection uses the retained extracted project.

Handover: retain the original ZIP, back up the SQLite database before seeding, preserve compatible artifact files together, and use the numbered pages of this document to trace each input and output to its implementation.